SELinux Configuration & Access Denial

Practical Assignment - Questions and Answers

Name	Sujal Chouhan
Task Name	SELinux Enforcing Mode and Web Server Context Troubleshooting
Objective	Check SELinux status, enable Enforcing mode, reproduce an access denial, correct the file context, and document verification.

Task 1 - Check the Current SELinux Status

Question: Use `getenforce` and `sestatus` to find out whether SELinux is Enforcing, Permissive, or Disabled.

Answer: Run the commands below as root or with sudo, then record the output from your own server.

```
getenforce
sestatus
```

How to record the result: `getenforce` prints one word: Enforcing, Permissive, or Disabled. The `sestatus` command gives more details, including the current mode and the mode configured in the SELinux configuration file.

> Lab note: SELinux was reported as Disabled in an earlier check for this lab. Re-run both commands now and write down the current output; do not assume the status is unchanged.

Task 2 - Switch SELinux to Enforcing Mode

Question: Use `setenforce` to switch SELinux to Enforcing and verify using `getenforce`.

Answer - if the current mode is Permissive:

```
setenforce 1
getenforce
# Expected output: Enforcing
```

The command `setenforce 1` changes the running SELinux mode to Enforcing immediately. This works when SELinux is already enabled (for example, Permissive mode).

> Important - if the current mode is Disabled: `setenforce 1` cannot enable SELinux while it is Disabled. Configure SELinux to start in Enforcing mode and reboot. This needs root/sudo privileges.

```
vi /etc/selinux/config
# Set this line: SELINUX=enforcing
# Keep/set: SELINUXTYPE=targeted
touch /.autorelabel
reboot
```

After reboot, file relabeling may take some time. Log in and verify with `getenforce` and `sestatus`. Expected current mode: Enforcing.

Task 3 - Simulate an SELinux Access Denial

Question: Create a test web page, label it with a type that the web server is not allowed to read (such as `user_home_t`), request the page, and inspect the audit log.

Answer: The steps below use Apache (httpd) and its default web directory `/var/www/html`. This is a reliable test location because it is already configured as the default DocumentRoot on a standard RHEL installation.

Step A - Install/start the required services

```
dnf install -y httpd policycoreutils-python-utils audit
systemctl enable --now httpd
systemctl enable --now auditd
```

Step B - Create a simple test page

```
mkdir -p /var/www/html/selinux-test
echo "SELinux test page" > /var/www/html/selinux-test/index.html
chmod 755 /var/www/html/selinux-test
chmod 644 /var/www/html/selinux-test/index.html
```

Step C - Check the current context, then apply the intentionally incorrect type

```
ls -Zd /var/www/html/selinux-test
chcon -R -t user_home_t /var/www/html/selinux-test
ls -Zd /var/www/html/selinux-test
```

The chcon command changes the SELinux type to `user_home_t`. This is intentionally unsuitable for static web content served by Apache, whose worker process normally runs in the `httpd_t` domain.

Step D - Request the page and check the response

```
curl -i http://localhost/selinux-test/index.html
```

Expected result: the request may return HTTP/1.1 403 Forbidden because SELinux blocks access. If the page still loads, verify that the server is running in Enforcing mode and confirm the file/directory label with `ls -Z`.

Step E - Find the SELinux denial in the audit log

```
ausearch -m AVC,USER_AVC -ts recent
grep -i "avc:.*denied" /var/log/audit/audit.log | tail -n 10
```

Example of the kind of log entry to look for (illustrative only; your PID, timestamp, path, and permissions will differ):

```
type=AVC ... avc: denied { open } for pid=... comm="httpd" \
path="/var/www/html/selinux-test/index.html" \
scontext=system_u:system_r:httpd_t:s0 \
tcontext=system_u:object_r:user_home_t:s0 tclass=file permissive=0
```

This record means the httpd process (source type/domain `httpd_t`) was denied access to a file labeled `user_home_t`. Use the actual event from your own audit log in your submitted evidence.

Task 4 - Correct the SELinux Context

Question: Use `chcon` to assign `httpd_sys_content_t` to the test content, then reload the page and confirm the issue is resolved.

Answer: Change the context type from `user_home_t` to `httpd_sys_content_t`, which is the standard SELinux type for static files served by Apache.

```
chcon -R -t httpd_sys_content_t /var/www/html/selinux-test
ls -Z /var/www/html/selinux-test/index.html
curl -i http://localhost/selinux-test/index.html
```

Expected verification: the response is successful (HTTP 200 OK) and the page body contains SELinux test page. The file label should include `httpd_sys_content_t`.

Optional - make the label persistent: `chcon` changes the current label but may be overwritten by a later relabel or `restorecon`. For a persistent rule, use `semanage fcontext` and then `restorecon`.

```
semanage fcontext -a -t httpd_sys_content_t \
  "/var/www/html/selinux-test(/.*)?"
restorecon -Rv /var/www/html/selinux-test
```

Task 5 - Document the Steps and Explain SELinux Contexts

Question: Document how the denial was found and resolved, include commands and a log entry, and explain how contexts control access.

Answer / summary:

1. I checked the SELinux mode with `getenforce` and `sestatus`. I made sure SELinux was enabled in Enforcing mode before testing access control.

2. I created a static HTML test page under `/var/www/html/selinux-test` and changed its type to `user_home_t` with chcon.

3. I requested the page using `curl` and inspected `/var/log/audit/audit.log` with `ausearch` or `grep` to find the AVC denial.

4. I changed the type to `httpd_sys_content_t` and tested the page again. The web server could then read and return the static page.

Explanation: SELinux applies mandatory access control using security contexts. A context contains fields such as user, role, type, and level. Apache normally runs in the `httpd_t` domain; SELinux policy permits it to read appropriately labeled web content such as `httpd_sys_content_t`, but it does not generally permit arbitrary files labeled `user_home_t`. Correct Unix permissions alone do not override an SELinux denial.

Final Verification Checklist

Check	Expected result
SELinux status	`getenforce` shows Enforcing after setup
Web server	`systemctl is-active httpd` returns active
Incorrect label test	AVC denial appears in audit log while type is `user_home_t`
Correct label	Test file label includes `httpd_sys_content_t`
Final HTTP test	`curl` returns HTTP 200 and page content

Evidence to attach: screenshots of `getenforce`/`sestatus`, the denial test, the AVC audit entry, `ls -Z` before and after the fix, and successful `curl` output. Replace illustrative output in this assignment with actual output from your server.